AIOps 통합

개요

AI 기반 운영 자동화를 단계적으로 구현합니다.

```
Phase 1: 알림 분석 + 제안 (현재 목표)
    ↓
Phase 2: 승인 기반 자동 실행
    ↓
Phase 3: 완전 자동화 (위험도 낮은 작업)
```

아키텍처

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Monitoring Layer                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐               │
│  │ Prometheus   │  │    Loki      │  │   K8s API    │               │
│  │  (Metrics)   │  │   (Logs)     │  │  (Events)    │               │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘               │
└─────────┼─────────────────┼─────────────────┼──────────────────────┘
          │                 │                 │
          ▼                 ▼                 ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        Alertmanager                                  │
│  - Alert Routing                                                     │
│  - Grouping                                                          │
│  - Silencing                                                         │
└──────────────────────────┬──────────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        AI Agent Service                              │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │                      Alert Receiver                      │       │
│  └──────────────────────────┬───────────────────────────────┘       │
│                             │                                        │
│  ┌──────────────────────────▼───────────────────────────────┐       │
│  │                  Context Collector                       │       │
│  │  - Related metrics                                       │       │
│  │  - Recent logs                                           │       │
│  │  - K8s events                                            │       │
│  │  - Historical incidents                                  │       │
│  └──────────────────────────┬───────────────────────────────┘       │
│                             │                                        │
│  ┌──────────────────────────▼───────────────────────────────┐       │
│  │                  LLM Analysis (Claude API)               │       │
│  │  - Root cause analysis                                   │       │
```

```
│   │  - Suggested actions                                    │    │
│   │  - Risk assessment                                      │    │
│   └─────────────────────────────────────────────────────────┘    │
│                              │                                    │
│                              │                                    │
│   ┌──────────────────────────▼──────────────────────────────┐    │
│   │                Action Engine                            │    │
│   │  - Phase 1: Notify (Slack/Discord)                      │    │
│   │  - Phase 2: Request approval + Execute                  │    │
│   │  - Phase 3: Auto-execute (low risk)                     │    │
│   └─────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────┘
```

Phase 1: 알림 분석 + 제안

AI Agent 서비스 구현 (Python)

```python
# ai_agent/main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Optional
import httpx
import anthropic
from datetime import datetime, timedelta
import json

app = FastAPI()
client = anthropic.Anthropic()

class Alert(BaseModel):
    status: str
    labels: dict
    annotations: dict
    startsAt: str
    endsAt: Optional[str]
    generatorURL: Optional[str]

class AlertmanagerWebhook(BaseModel):
    version: str
    groupKey: str
    status: str
    receiver: str
    alerts: List[Alert]

class ContextCollector:
    def __init__(self, prometheus_url: str, loki_url: str):
        self.prometheus_url = prometheus_url
        self.loki_url = loki_url
        self.http_client = httpx.AsyncClient(timeout=30.0)

    async def get_related_metrics(self, alert: Alert) -> dict:
        """알림과 관련된 메트릭 수집"""
```

```
        namespace = alert.labels.get("namespace", "")
        pod = alert.labels.get("pod", "")

        queries = {
            "cpu_usage": f'rate(container_cpu_usage_seconds_total{{namespace="
{namespace}",pod=~"{pod}.*"}}[5m])',
            "memory_usage": f'container_memory_usage_bytes{{namespace="
{namespace}",pod=~"{pod}.*"}}',
            "restarts": f'kube_pod_container_status_restarts_total{{namespace="
{namespace}",pod=~"{pod}.*"}}',
        }

        # LLM/블록체인 특화 메트릭
        if "vllm" in pod or "llm" in pod:
            queries.update({
                "queue_length": f'vllm_num_requests_waiting{{namespace="
{namespace}"}}',
                "gpu_util": f'DCGM_FI_DEV_GPU_UTIL{{namespace="{namespace}"}}',
                "inference_latency": f'histogram_quantile(0.99,
rate(vllm_request_latency_seconds_bucket{{namespace="{namespace}"}}[5m]))',
            })
        elif "cosmos" in pod or "tendermint" in pod or "geth" in pod:
            queries.update({
                "block_height": f'tendermint_consensus_height{{namespace="
{namespace}"}}',
                "peers": f'tendermint_p2p_peers{{namespace="{namespace}"}}',
                "sync_lag": f'max(tendermint_consensus_height) -
tendermint_consensus_height{{namespace="{namespace}"}}',
            })

        results = {}
        for name, query in queries.items():
            try:
                resp = await self.http_client.get(
                    f"{self.prometheus_url}/api/v1/query",
                    params={"query": query}
                )
                data = resp.json()
                if data["data"]["result"]:
                    results[name] = data["data"]["result"]
            except Exception as e:
                results[name] = f"Error: {str(e)}"

        return results

    async def get_recent_logs(self, alert: Alert, minutes: int = 15) -> str:
        """최근 로그 수집"""
        namespace = alert.labels.get("namespace", "")
        pod = alert.labels.get("pod", "")

        end_time = datetime.utcnow()
        start_time = end_time - timedelta(minutes=minutes)

        query = f'{{namespace="{namespace}",pod=~"{pod}.*"}}'
```

```
        try:
            resp = await self.http_client.get(
                f"{self.loki_url}/loki/api/v1/query_range",
                params={
                    "query": query,
                    "start": int(start_time.timestamp() * 1e9),
                    "end": int(end_time.timestamp() * 1e9),
                    "limit": 100,
                }
            )
            data = resp.json()

            logs = []
            for stream in data.get("data", {}).get("result", []):
                for value in stream.get("values", []):
                    logs.append(value[1])

            return "\n".join(logs[-50:])  # 최근 50줄
        except Exception as e:
            return f"Error fetching logs: {str(e)}"

    async def get_k8s_events(self, alert: Alert) -> List[dict]:
        """K8s 이벤트 수집"""
        # 실제 구현에서는 kubernetes client 사용
        namespace = alert.labels.get("namespace", "")
        return []


class AIAnalyzer:
    def __init__(self):
        self.client = anthropic.Anthropic()

    async def analyze(self, alert: Alert, context: dict) -> dict:
        """알림 분석 및 대응 방안 제안"""

        prompt = f"""당신은 Kubernetes 클러스터에서 LLM 서빙과 블록체인 노드를 관리하
는 SRE입니다.
다음 알림과 컨텍스트를 분석하고 대응 방안을 제안해주세요.

## 알림 정보
- 이름: {alert.labels.get('alertname', 'Unknown')}
- 심각도: {alert.labels.get('severity', 'Unknown')}
- 네임스페이스: {alert.labels.get('namespace', 'Unknown')}
- 요약: {alert.annotations.get('summary', 'N/A')}
- 설명: {alert.annotations.get('description', 'N/A')}
- 발생 시간: {alert.startsAt}

## 관련 메트릭
{json.dumps(context.get('metrics', {}), indent=2, default=str)}

## 최근 로그
```

{context.get('logs', 'No logs available')}

```
## 분석 요청
1. 근본 원인 분석 (Root Cause Analysis)
2. 즉각적인 대응 방안
3. 장기적인 예방 조치
4. 위험도 평가 (1-10)
5. 자동 실행 가능한 명령어 (있다면)

JSON 형식으로 응답해주세요:
{{
    "root_cause": "분석 결과",
    "immediate_actions": ["조치1", "조치2"],
    "preventive_measures": ["예방1", "예방2"],
    "risk_level": 5,
    "auto_executable_commands": [
        {{
            "description": "설명",
            "command": "kubectl ...",
            "risk": "low|medium|high"
        }}
    ],
    "summary": "한줄 요약"
}}
"""

        response = self.client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=2000,
            messages=[{"role": "user", "content": prompt}]
        )

        # JSON 파싱
        try:
            content = response.content[0].text
            # JSON 블록 추출
            if "```json" in content:
                content = content.split("```json")[1].split("```")[0]
            elif "```" in content:
                content = content.split("```")[1].split("```")[0]

            return json.loads(content)
        except json.JSONDecodeError:
            return {
                "root_cause": "분석 실패",
                "immediate_actions": ["수동 확인 필요"],
                "preventive_measures": [],
                "risk_level": 5,
                "auto_executable_commands": [],
                "summary": response.content[0].text[:200],
                "raw_response": response.content[0].text
            }
```

```
class NotificationSender:
    def __init__(self, slack_webhook: str = None, discord_webhook: str = None):
        self.slack_webhook = slack_webhook
        self.discord_webhook = discord_webhook
        self.http_client = httpx.AsyncClient()

    async def send_slack(self, alert: Alert, analysis: dict):
        """Slack으로 분석 결과 전송"""
        if not self.slack_webhook:
            return

        risk_emoji = "🔴" if analysis["risk_level"] <= 3 else "🟡" if
analysis["risk_level"] <= 6 else "🟢"

        blocks = [
            {
                "type": "header",
                "text": {
                    "type": "plain_text",
                    "text": f"{risk_emoji} {alert.labels.get('alertname',
'Alert')}"
                }
            },
            {
                "type": "section",
                "text": {
                    "type": "mrkdwn",
                    "text": f"*요약:* {analysis['summary']}"
                }
            },
            {
                "type": "section",
                "text": {
                    "type": "mrkdwn",
                    "text": f"*근본 원인:*\n{analysis['root_cause']}"
                }
            },
            {
                "type": "section",
                "text": {
                    "type": "mrkdwn",
                    "text": f"*즉각 조치:*\n" + "\n".join(f"• {a}" for a in
analysis['immediate_actions'])
                }
            },
        ]

        # 자동 실행 가능한 명령어가 있으면 버튼 추가 (Phase 2용)
        if analysis.get("auto_executable_commands"):
            for cmd in analysis["auto_executable_commands"]:
                if cmd["risk"] == "low":
                    blocks.append({
```

```
                        "type": "actions",
                        "elements": [
                            {
                                "type": "button",
                                "text": {"type": "plain_text", "text": f"실행:
{cmd['description']}"},
                                "style": "primary",
                                "action_id": f"execute_{hash(cmd['command'])}",
                                "value": cmd["command"]
                            }
                        ]
                    })

        await self.http_client.post(
            self.slack_webhook,
            json={"blocks": blocks}
        )

    async def send_discord(self, alert: Alert, analysis: dict):
        """Discord로 분석 결과 전송"""
        if not self.discord_webhook:
            return

        risk_color = 0x00ff00 if analysis["risk_level"] <= 3 else 0xffff00 if
analysis["risk_level"] <= 6 else 0xff0000

        embed = {
            "title": f"🚨 {alert.labels.get('alertname', 'Alert')}",
            "color": risk_color,
            "fields": [
                {"name": "요약", "value": analysis["summary"], "inline": False},
                {"name": "근본 원인", "value": analysis["root_cause"][:1000],
"inline": False},
                {"name": "즉각 조치", "value": "\n".join(f"• {a}" for a in
analysis["immediate_actions"][:5]), "inline": False},
                {"name": "위험도", "value": f"{analysis['risk_level']}/10",
"inline": True},
            ]
        }

        await self.http_client.post(
            self.discord_webhook,
            json={"embeds": [embed]}
        )


# 전역 인스턴스
collector = ContextCollector(
    prometheus_url="http://prometheus.monitoring.svc:9090",
    loki_url="http://loki.monitoring.svc:3100"
)
analyzer = AIAnalyzer()
notifier = NotificationSender(
    slack_webhook=None,  # 환경변수에서 로드
```

```
    discord_webhook=None
)


@app.post("/webhook/alertmanager")
async def handle_alertmanager_webhook(payload: AlertmanagerWebhook):
    """Alertmanager 웹훅 수신"""

    results = []

    for alert in payload.alerts:
        if payload.status != "firing":
            continue  # resolved 알림은 스킵

        # 1. 컨텍스트 수집
        context = {
            "metrics": await collector.get_related_metrics(alert),
            "logs": await collector.get_recent_logs(alert),
            "events": await collector.get_k8s_events(alert),
        }

        # 2. AI 분석
        analysis = await analyzer.analyze(alert, context)

        # 3. 알림 전송
        await notifier.send_slack(alert, analysis)
        await notifier.send_discord(alert, analysis)

        results.append({
            "alert": alert.labels.get("alertname"),
            "analysis": analysis
        })

    return {"status": "processed", "results": results}


@app.get("/health")
async def health():
    return {"status": "ok"}
```

Dockerfile

```
# ai_agent/Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

requirements.txt

```
fastapi==0.109.0
uvicorn==0.27.0
httpx==0.26.0
anthropic==0.18.0
pydantic==2.5.0
```

Kubernetes 배포

```yaml
# manifests/aiops/ai-agent-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-agent
  namespace: monitoring
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-agent
  template:
    metadata:
      labels:
        app: ai-agent
    spec:
      containers:
      - name: ai-agent
        image: your-registry/ai-agent:latest
        ports:
        - containerPort: 8080
        env:
        - name: ANTHROPIC_API_KEY
          valueFrom:
            secretKeyRef:
              name: ai-agent-secrets
              key: anthropic-api-key
        - name: SLACK_WEBHOOK_URL
          valueFrom:
            secretKeyRef:
              name: ai-agent-secrets
              key: slack-webhook
        - name: PROMETHEUS_URL
          value: "http://prometheus.monitoring.svc:9090"
        - name: LOKI_URL
          value: "http://loki.monitoring.svc:3100"
```

```yaml
        resources:
          requests:
            cpu: "200m"
            memory: "256Mi"
          limits:
            cpu: "1"
            memory: "512Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: ai-agent
  namespace: monitoring
spec:
  selector:
    app: ai-agent
  ports:
  - port: 80
    targetPort: 8080
```

Alertmanager 설정

```yaml
# manifests/monitoring/alertmanager-config.yaml
apiVersion: v1
kind: Secret
metadata:
  name: alertmanager-config
  namespace: monitoring
stringData:
  alertmanager.yml: |
    global:
      resolve_timeout: 5m

    route:
      group_by: ['alertname', 'namespace']
      group_wait: 30s
      group_interval: 5m
      repeat_interval: 4h
      receiver: 'ai-agent'

      routes:
      # Critical 알림은 즉시 처리
      - match:
          severity: critical
        group_wait: 10s
        receiver: 'ai-agent'

      # Warning은 그룹화해서 처리
      - match:
          severity: warning
        group_wait: 1m
```

```
        receiver: 'ai-agent'

    receivers:
    - name: 'ai-agent'
      webhook_configs:
      - url: 'http://ai-agent.monitoring.svc/webhook/alertmanager'
        send_resolved: true
```

Phase 2: 승인 기반 자동 실행

Action Executor 추가

```
# ai_agent/executor.py
from kubernetes import client, config
from kubernetes.client.rest import ApiException
import asyncio
from typing import Optional
import logging

logger = logging.getLogger(__name__)

class ActionExecutor:
    def __init__(self):
        try:
            config.load_incluster_config()
        except:
            config.load_kube_config()

        self.core_v1 = client.CoreV1Api()
        self.apps_v1 = client.AppsV1Api()

        # 허용된 액션 목록 (화이트리스트)
        self.allowed_actions = {
            "restart_pod": self.restart_pod,
            "scale_deployment": self.scale_deployment,
            "delete_pod": self.delete_pod,
            "cordon_node": self.cordon_node,
        }

    async def execute(self, action: str, params: dict, approved_by: Optional[str]
= None) -> dict:
        """승인된 액션 실행"""

        if action not in self.allowed_actions:
            return {"success": False, "error": f"Unknown action: {action}"}

        # 감사 로그
        logger.info(f"Executing action: {action}, params: {params}, approved_by:
{approved_by}")

        try:
```

```
            result = await self.allowed_actions[action](**params)
            return {"success": True, "result": result}
        except ApiException as e:
            logger.error(f"K8s API error: {e}")
            return {"success": False, "error": str(e)}
        except Exception as e:
            logger.error(f"Execution error: {e}")
            return {"success": False, "error": str(e)}

    async def restart_pod(self, namespace: str, pod_name: str) -> str:
        """Pod 재시작 (삭제 후 자동 재생성)"""
        self.core_v1.delete_namespaced_pod(
            name=pod_name,
            namespace=namespace
        )
        return f"Pod {pod_name} deleted for restart"

    async def scale_deployment(self, namespace: str, deployment: str, replicas:
int) -> str:
        """Deployment 스케일 조정"""
        body = {"spec": {"replicas": replicas}}
        self.apps_v1.patch_namespaced_deployment_scale(
            name=deployment,
            namespace=namespace,
            body=body
        )
        return f"Scaled {deployment} to {replicas} replicas"

    async def delete_pod(self, namespace: str, pod_name: str) -> str:
        """Pod 삭제"""
        self.core_v1.delete_namespaced_pod(
            name=pod_name,
            namespace=namespace
        )
        return f"Pod {pod_name} deleted"

    async def cordon_node(self, node_name: str) -> str:
        """노드 cordon (스케줄링 비활성화)"""
        body = {
            "spec": {
                "unschedulable": True
            }
        }
        self.core_v1.patch_node(name=node_name, body=body)
        return f"Node {node_name} cordoned"


# Slack 버튼 핸들러 (Phase 2)
from fastapi import Request

executor = ActionExecutor()

@app.post("/slack/interactive")
async def handle_slack_interactive(request: Request):
```

```
    """Slack 버튼 클릭 처리"""
    form = await request.form()
    payload = json.loads(form["payload"])

    action = payload["actions"][0]
    user = payload["user"]["username"]

    if action["action_id"].startswith("execute_"):
        command = action["value"]

        # 명령어 파싱 및 실행
        # 실제로는 더 정교한 파싱 필요
        if "delete pod" in command:
            parts = command.split()
            result = await executor.execute(
                "delete_pod",
                {"namespace": parts[-2], "pod_name": parts[-1]},
                approved_by=user
            )
        elif "scale" in command:
            # scale deployment 파싱
            result = {"success": False, "error": "Not implemented"}
        else:
            result = {"success": False, "error": "Unknown command"}

        return {
            "response_type": "in_channel",
            "text": f"실행 결과: {result}"
        }

    return {"status": "ok"}
```

RBAC 설정

```yaml
# manifests/aiops/ai-agent-rbac.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ai-agent
  namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: ai-agent-role
rules:
# Pod 관리 (제한적)
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch", "delete"]
# Deployment 스케일링
```

```yaml
  - apiGroups: ["apps"]
    resources: ["deployments/scale"]
    verbs: ["get", "patch"]
  # 노드 cordon
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list", "patch"]
  # 이벤트/로그 조회
  - apiGroups: [""]
    resources: ["events", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: ai-agent-binding
subjects:
- kind: ServiceAccount
  name: ai-agent
  namespace: monitoring
roleRef:
  kind: ClusterRole
  name: ai-agent-role
  apiGroup: rbac.authorization.k8s.io
```

Phase 3: 자동 실행 (위험도 기반)

```python
# ai_agent/auto_executor.py

class AutoExecutionPolicy:
    """위험도 기반 자동 실행 정책"""

    # 자동 실행 허용 조건
    AUTO_EXECUTE_CONDITIONS = {
        "restart_pod": {
            "max_risk_level": 3,
            "allowed_namespaces": ["customer-*"],  # 고객 네임스페이스만
            "cooldown_minutes": 30,  # 30분 내 중복 실행 방지
            "max_per_hour": 5,
        },
        "scale_deployment": {
            "max_risk_level": 2,
            "allowed_namespaces": ["customer-*"],
            "scale_limits": {"min": 1, "max": 10},
            "cooldown_minutes": 15,
        },
    }

    def __init__(self, redis_client):
        self.redis = redis_client
```

```
    async def can_auto_execute(self, action: str, params: dict, risk_level: int) -
> tuple[bool, str]:
        """자동 실행 가능 여부 판단"""

        if action not in self.AUTO_EXECUTE_CONDITIONS:
            return False, f"Action {action} not allowed for auto-execution"

        policy = self.AUTO_EXECUTE_CONDITIONS[action]

        # 위험도 체크
        if risk_level > policy["max_risk_level"]:
            return False, f"Risk level {risk_level} exceeds threshold
{policy['max_risk_level']}"

        # 네임스페이스 체크
        namespace = params.get("namespace", "")
        if not any(fnmatch.fnmatch(namespace, p) for p in
policy["allowed_namespaces"]):
            return False, f"Namespace {namespace} not in allowed list"

        # 쿨다운 체크
        cooldown_key = f"cooldown:{action}:{namespace}:{params.get('pod_name',
params.get('deployment', ''))}"
        if await self.redis.exists(cooldown_key):
            return False, "Action in cooldown period"

        # 시간당 횟수 체크
        rate_key = f"rate:{action}:{namespace}"
        current_rate = await self.redis.incr(rate_key)
        if current_rate == 1:
            await self.redis.expire(rate_key, 3600)
        if current_rate > policy["max_per_hour"]:
            return False, f"Rate limit exceeded ({policy['max_per_hour']}/hour)"

        return True, "OK"

    async def record_execution(self, action: str, params: dict):
        """실행 기록 저장"""
        policy = self.AUTO_EXECUTE_CONDITIONS.get(action, {})
        cooldown_key = f"cooldown:{action}:{params.get('namespace', '')}:
{params.get('pod_name', params.get('deployment', ''))}"
        await self.redis.setex(cooldown_key, policy.get("cooldown_minutes", 30) *
60, "1")
```

모니터링 및 감사

AI Agent 메트릭

```
# ai_agent/metrics.py
from prometheus_client import Counter, Histogram, Gauge
```

```
alerts_received = Counter(
    'aiops_alerts_received_total',
    'Total alerts received',
    ['alertname', 'severity', 'namespace']
)

analysis_duration = Histogram(
    'aiops_analysis_duration_seconds',
    'Time spent on alert analysis',
    ['alertname']
)

actions_executed = Counter(
    'aiops_actions_executed_total',
    'Total actions executed',
    ['action', 'namespace', 'auto_executed', 'success']
)

pending_approvals = Gauge(
    'aiops_pending_approvals',
    'Number of actions pending approval',
    ['action']
)
```

감사 로그 스키마

```
-- 감사 로그 테이블 (PostgreSQL)
CREATE TABLE aiops_audit_log (
    id SERIAL PRIMARY KEY,
    timestamp TIMESTAMPTZ DEFAULT NOW(),

    -- 알림 정보
    alert_name VARCHAR(255),
    alert_severity VARCHAR(50),
    namespace VARCHAR(255),

    -- AI 분석
    analysis_summary TEXT,
    risk_level INTEGER,
    root_cause TEXT,

    -- 실행 정보
    action_type VARCHAR(100),
    action_params JSONB,
    auto_executed BOOLEAN,
    approved_by VARCHAR(255),

    -- 결과
    execution_result JSONB,
    success BOOLEAN,
    error_message TEXT
```

```
);

CREATE INDEX idx_audit_timestamp ON aiops_audit_log(timestamp);
CREATE INDEX idx_audit_namespace ON aiops_audit_log(namespace);
CREATE INDEX idx_audit_action ON aiops_audit_log(action_type);
```

확장 가능한 질문들

다음 단계 고려사항

1. **학습 시스템**: 과거 인시던트와 해결 방법을 학습하여 분석 정확도 향상
2. **예측 기능**: 메트릭 추세 분석으로 장애 예측
3. **비용 최적화**: 사용량 분석으로 리소스 최적화 제안
4. **멀티 클러스터**: 여러 클러스터 통합 관리
5. **고객별 커스터마이징**: 고객별 알림 규칙/대응 정책

다음: TODO.md - 구현 체크리스트